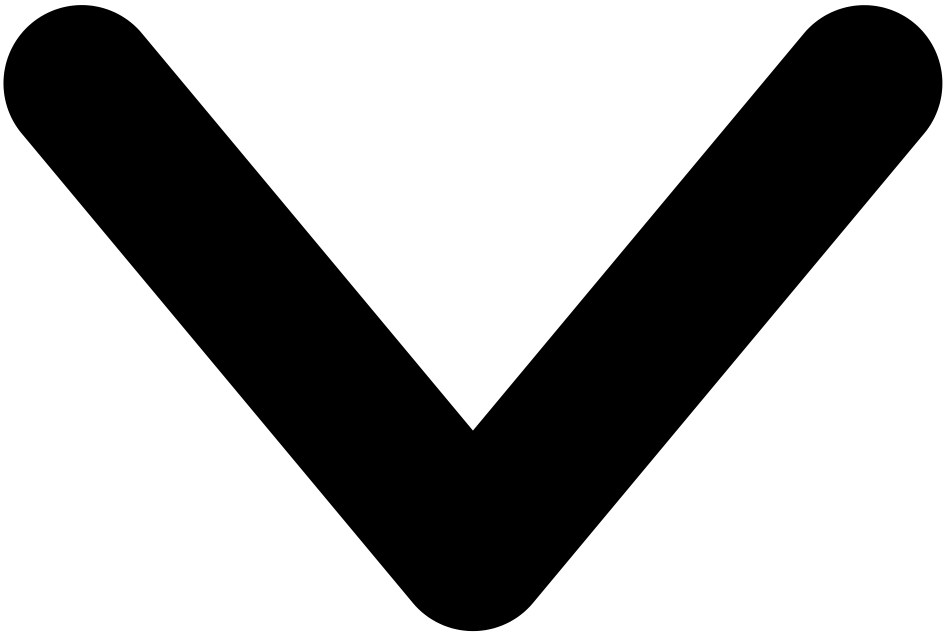


Suppose you've just bought a Mac—or you're thinking about it—but macOS feels alien and confusing. You've probably heard the rumor that you can't so much as sneeze without Tim Cook's permission. Another common myth is that macOS is just a lightly reworked Linux. In this article, we'll walk through macOS's core mechanisms and also talk about what the real restrictions are and whether they can be worked around.

A Brief History of macOS

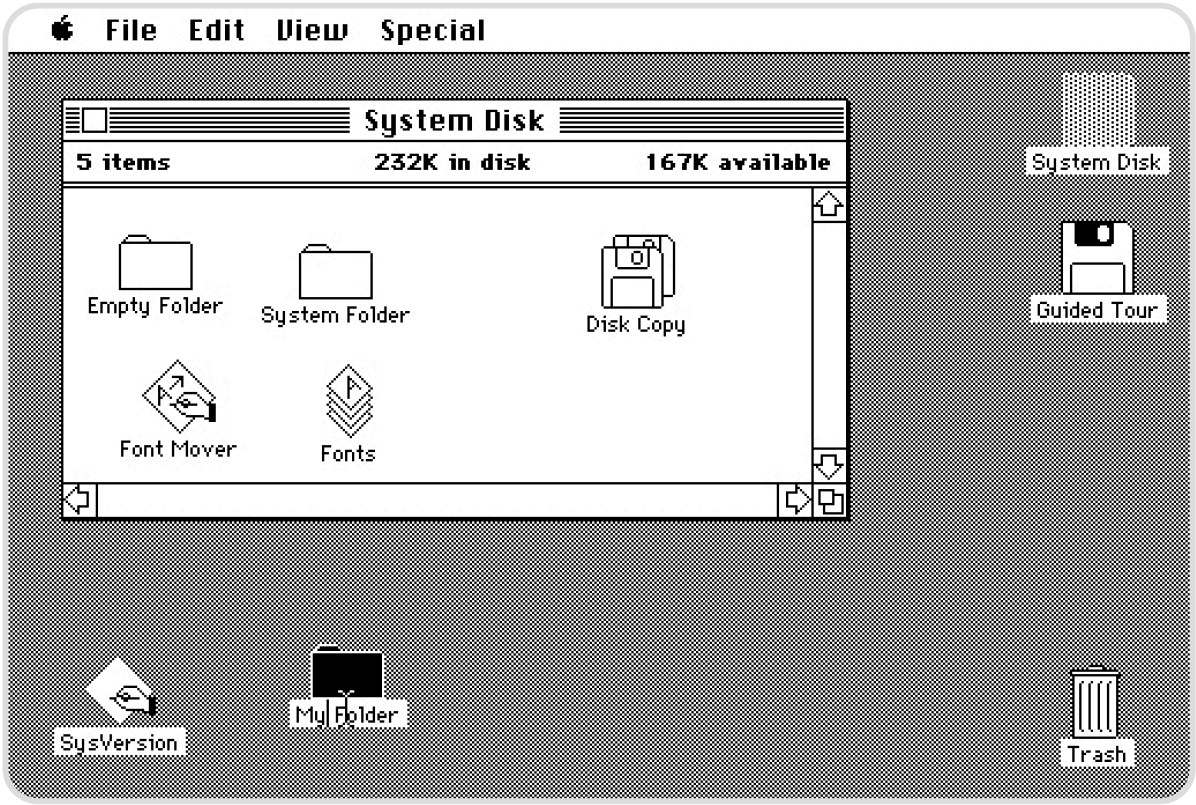
The history of macOS—like Apple's history as a whole—is fascinating and full of twists and turns. Here, I'll recount it in a very brief, simplified form.

It all started back in the 1980s with the Apple II. They didn't really have an operating system in the modern sense; what they had would today be considered firmware. Like other home computers of the era, it included a BASIC interpreter used to execute user commands.

WWW

There's no visible Apple II or III legacy left in macOS today, but if you want a taste of history you can run an **Apple II emulator** right in your browser.

The Apple Macintosh, released in 1984, was dramatically different from those machines. Its operating system came with a graphical user interface and mouse support right out of the box. A windowed interface was a striking novelty at the time—no mass-produced computer had it before then (Windows 1.0 arrived two years after the Macintosh and borrowed a lot from it).



One of the early versions of Mac OS

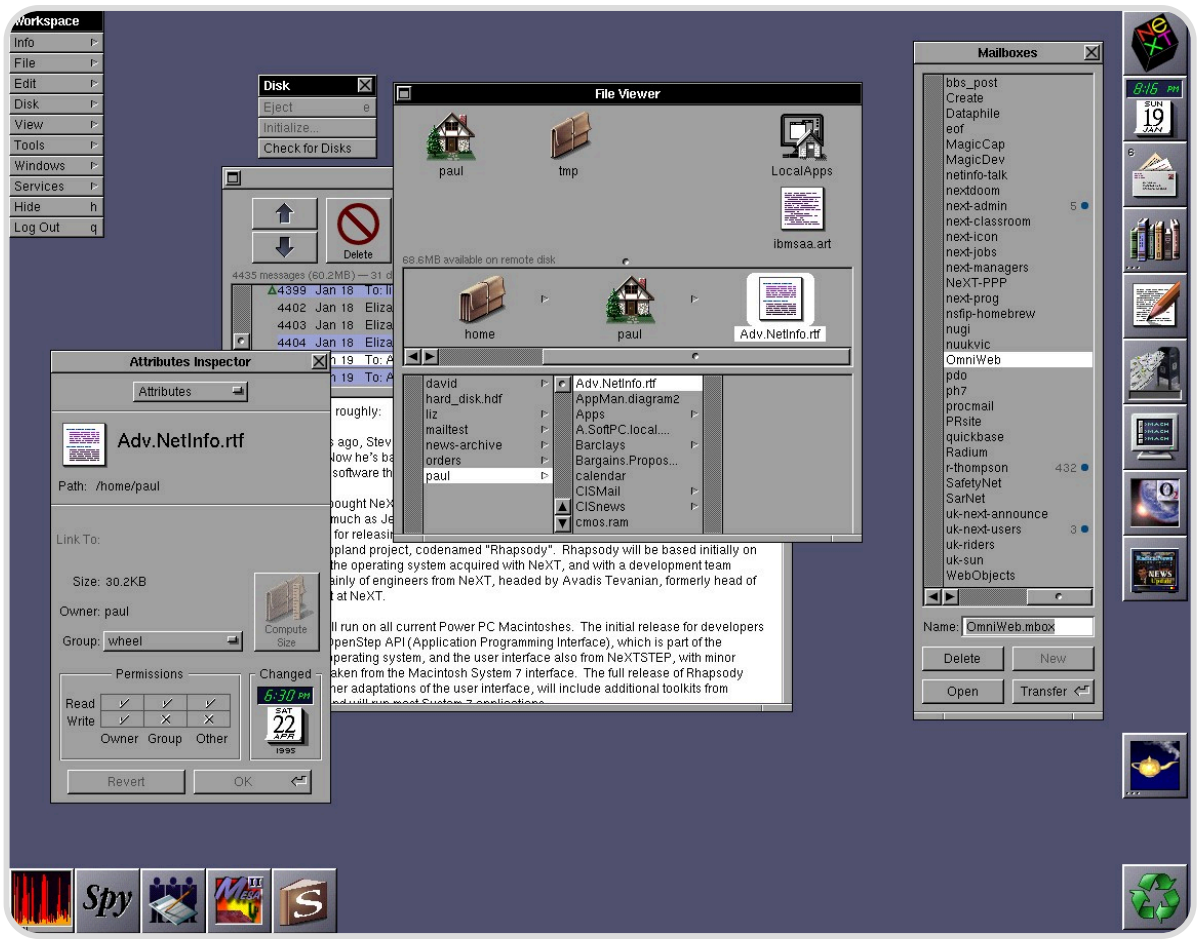
Classic Mac OS saw active development until 1996, with its final release shipping in 2001. While it was considered cutting-edge in the late '80s, by the '90s its architecture—with an outdated memory model—was

increasingly holding Apple back. As an emergency measure, the company's leadership decided to acquire NeXT, a startup founded by Steve Jobs after he was pushed out of Apple.



Mac OS 9, the last major release of the classic Mac OS

NeXT's flagship product was the NeXTSTEP graphical operating system, built on a Unix-like kernel and environment, an advanced graphics engine, and a set of object-oriented frameworks. Those frameworks let developers easily build GUI applications in Objective-C, which was cutting-edge at the time. For example, the prototype of the first web browser was created on NeXT machines.



NeXTSTEP

After the NeXT engineering team joined Apple, the two groups developed a new system: Mac OS X. It was later renamed OS X, and then macOS (Apple's marketing department never rests). Technically, Mac OS X is based on NeXTSTEP, but its interface drew heavily from the classic Mac OS.



Mac OS X 10.1 Cheetah

During the transition period, Macintosh systems supported both the classic Mac OS and Mac OS X. Starting in 2002, all Apple computers shipped with Mac OS X preinstalled, and Mac OS 9 could still be run for several more years in a compatibility mode.

The XNU Kernel

At the heart of macOS—like any other operating system—is the kernel. It's responsible for CPU scheduling, managing RAM and caches, and interfacing with hardware devices and the network. It also handles application system calls and facilitates interprocess communication (IPC).

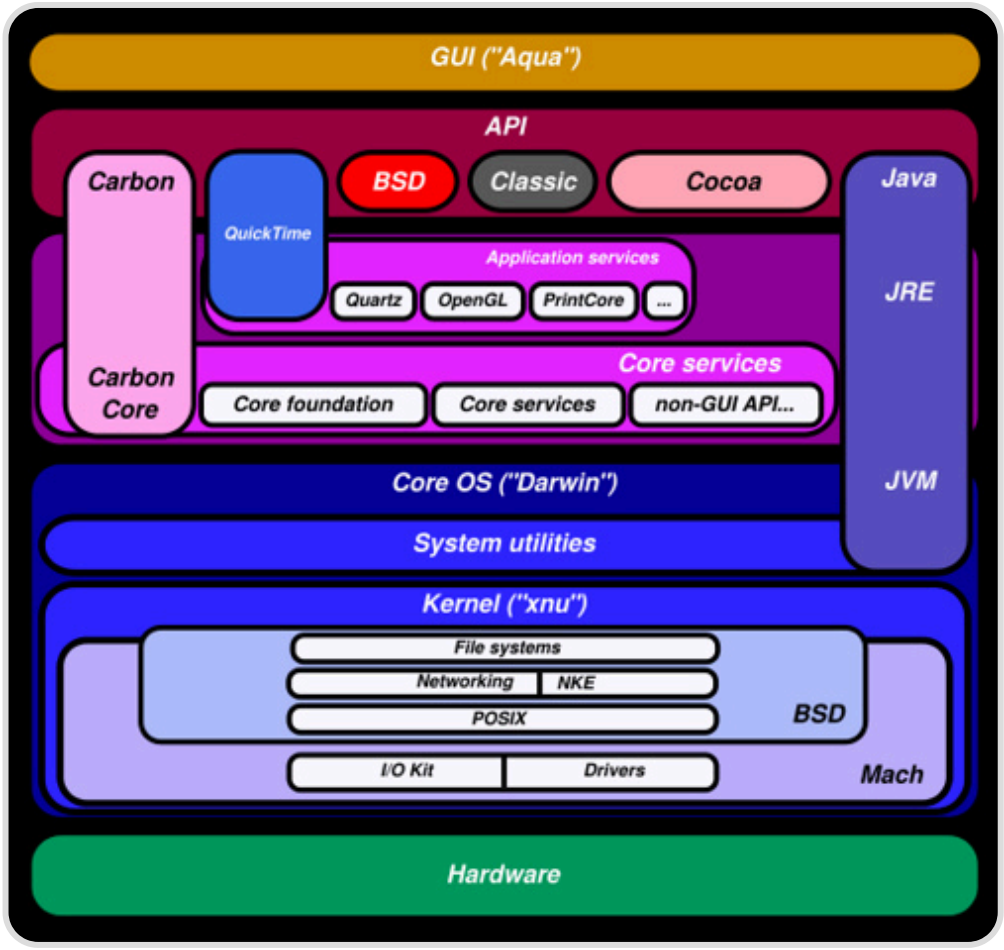
Modern macOS runs on the XNU kernel, inherited from NeXTSTEP. Its code is based on the Mach microkernel with a BSD layer that was originally forked from FreeBSD.

XNU stands for “X is Not Unix.” That expansion is dated programmer humor—macOS is broadly considered a UNIX variant today. That said, XNU is not binary-compatible with FreeBSD, so FreeBSD programs won’t run on macOS without modification and recompilation.

XNU is a hybrid kernel. In other words, unlike microkernels it can be extended with additional modules, yet it isn’t monolithic like the Linux kernel, where all functionality is bundled into a single giant binary.

Until macOS 10.15, the main way to extend the kernel was through kernel extensions (kexts). Because kexts run in kernel space, bugs in them can make the whole system unstable. They also created ample opportunities for abuse by unscrupulous developers.

“Kexts” are now considered deprecated and will be phased out over time. Instead, Apple recommends that developers use the DriverKit and System Extensions frameworks, which let you build drivers and extensions that run in user space.



macOS architecture

Darwin

An operating system is more than just a kernel. Along with Mach, a set of libraries and executables made its way into NeXTSTEP and later into Mac OS X; together with XNU they provide support for POSIX—the Portable Operating System Interface. This is a standard to which, to varying degrees, all Unix-like operating systems conform, enabling low-level compatibility among them.

In macOS, this layer is called Darwin and is essentially a standalone operating system. It doesn't include the graphical environment or the libraries needed for windowed applications, but it does include the kernel, drivers, network stack, a set of system and user command-line utilities, and the launchd system for starting services and applications.

If you want, you can install Darwin as a standalone minimalist OS with a text-based command interpreter. Darwin's code was open from the start, but over time it accumulated many closed components, including Mac-specific drivers.

By the last Darwin releases, you could no longer build and run it without tooling available only to Apple engineers. In effect, publishing the sources in that state wasn't useful to Apple or the community, so they simply stopped. The XNU code, meanwhile, is still **available** on GitHub and continues to be updated.

The community currently maintains the **PureDarwin** project—a truly open implementation of Darwin.

> **INFO**
> For a long time, advanced Mac users favored MacPorts—a suite of tools built for Darwin that packaged and updated modern Linux software for macOS. MacPorts is still maintained, but these days it's been largely supplanted by the Homebrew (brew) package manager.

The Graphics System

The graphics layer in macOS is commonly referred to as Quartz, though what's really meant is the Core Graphics framework. Its two most important components are Quartz 2D and the Quartz Compositor.

Quartz 2D

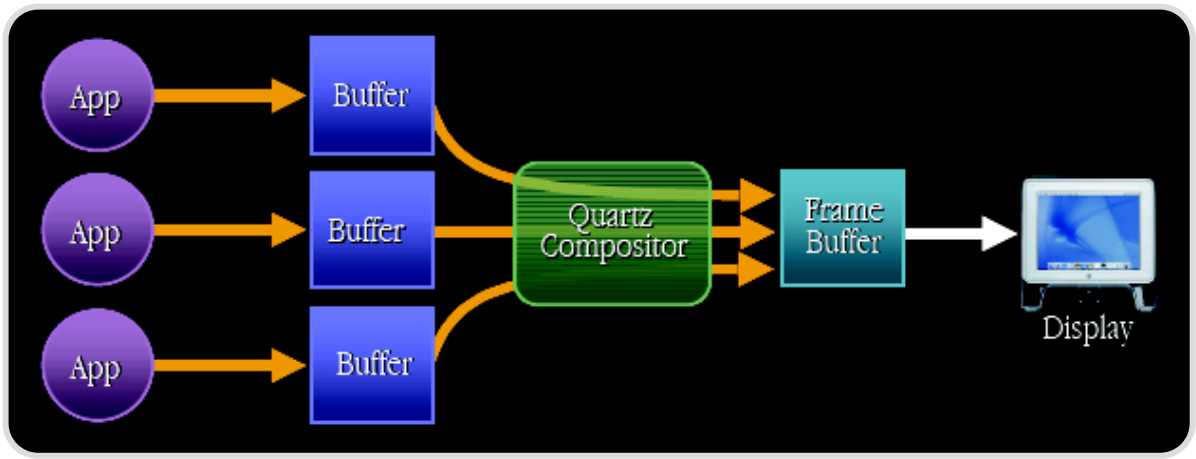
Quartz 2D handles everything related to two-dimensional graphics. Its core tasks include rendering text and converting PostScript-defined

graphic primitives into raster images, which are then handed off to the Quartz Compositor.

PostScript was originally developed at Adobe and designed for printing documents. It also became the foundation of the PDF format and, crucially for our purposes, was used on NeXT computers to describe on-screen graphics—which is how it made its way into macOS. Thanks to this close relationship between PostScript and PDF, Macs and iPhones handle PDFs natively, without needing Adobe Acrobat or any other viewer.

Quartz Compositor

Quartz Compositor serves both as a graphics server and a compositing window manager. On one hand, it receives bitmap images of application windows and composes them into the final frame shown on the screen; on the other, it handles input device events like mouse clicks and key presses. Quartz Compositor determines which window those events target and forwards them to the appropriate applications.



Since Quartz Compositor also acts as a window manager, it's responsible for rendering the user interface. That includes fonts, menus, window frames, panels, and other widgets collectively known as the Aqua interface. In early versions of Mac OS X, some elements really did resemble drops of water, but the design has been revamped many times since, and now it's just a name.

A different compositor

Don’t confuse Quartz Compositor with the development tool called Quartz Composer. Composer is a visual programming language with in the Xcode IDE that lets you assemble complex image and video tr ansformations from prebuilt blocks. This is the context in which the t erm “Quartz” sometimes shows up in graphics editors that can direc tly leverage Quartz Composer’s capabilities.

XQuartz

If you poke around the macOS developer tools, you can dig up another interesting artifact: XQuartz. It’s an implementation of the Unix X Window System display server. For a long time, its open-source implementation, X.Org, shipped with virtually every Linux distribution and was the de facto standard. Today, the venerable—if creaky—X is gradually being replaced by systems like Wayland and Mir, which in some respects are similar in design to macOS’s Quartz Compositor.



XQuartz

As for XQuartz, it was clearly something Apple put together with the idea of letting users run Unix and Linux GUI applications.

The demand turned out to be modest, though, and XQuartz never really evolved. On Linux, most graphical software depends on the GNOME or KDE libraries, and Apple never planned to port those to the Mac. As a result, XQuartz is really only useful for the simplest graphical programs that talk directly to the X Window System.

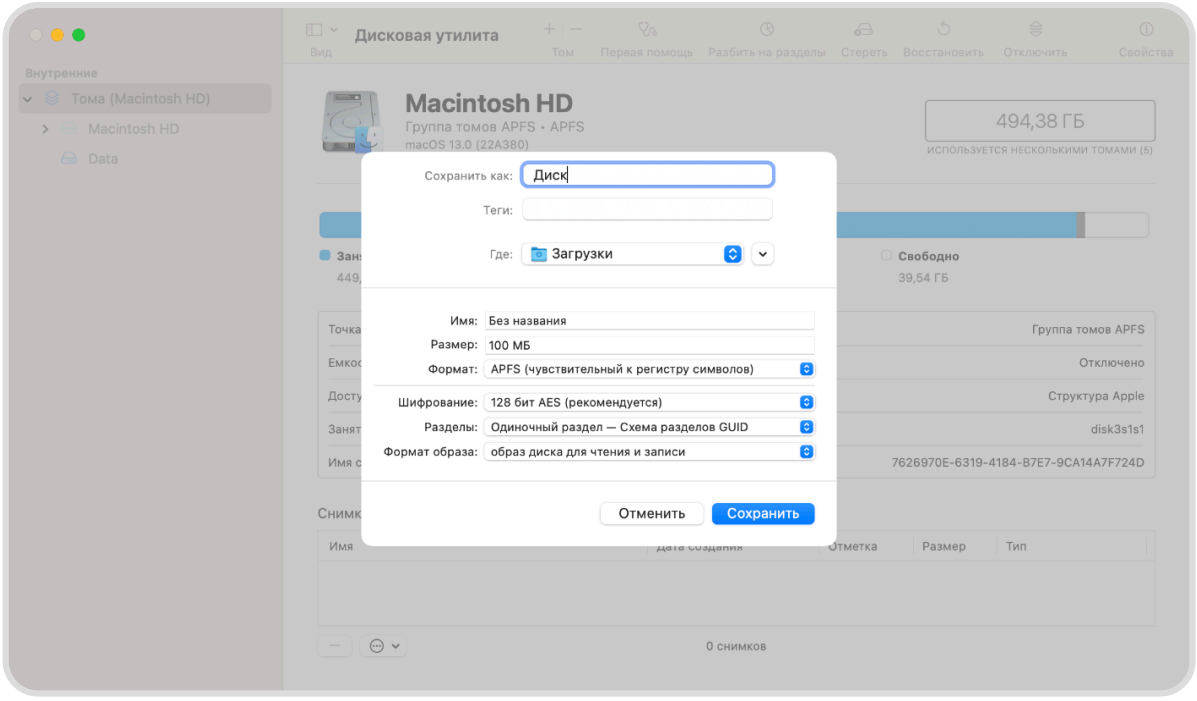
APFS

Until macOS 10.13 High Sierra, the default file system on every Mac was HFS+, inherited from the classic Mac OS. At the time, it was considered cutting-edge—HFS+, for example, supports Unicode and journaling (the filesystem journal is a log of operations that helps preserve data integrity in case of a crash or power loss). However, the original HFS was designed back in the 1980s and, despite the late-1990s upgrade to HFS+, it lacked support for many important modern features.

Apple developed a new file system called APFS (Apple File System) and successfully rolled it out to iOS in 2017. It offers a host of benefits that proved useful not only on iPhones and iPads, but on the Mac as well.

APFS was designed with solid-state drives (SSDs) in mind and, in particular, supports the TRIM command. It also uses 64-bit identifiers for files (instead of 32-bit in HFS+), supports sparse files (zeroed regions aren't physically allocated), can encrypt data at the file and directory level (not just the entire disk), and lets you share volumes over the network via multiple protocols.

APFS delivers a speed boost that's noticeable in everyday use. For example, calculating folder sizes and copying files is much faster than on HFS+.



Creating an APFS volume in Disk Utility

Finally, the most noticeable feature of APFS is that you don't have to pre-allocate a fixed size for logical volumes. They draw from a shared physical storage pool, and each one uses only as much space as its data requires.

The APFS specifications are open enough to enable the creation of read-only drivers. That said, there are already experimental Linux drivers that provide read/write support for APFS.

One important APFS trait users should know is that it's case-insensitive. In other words, `FILE`, `file`, `FiLe`, and similar variations are treated as the same name by APFS. You can enable case sensitivity when formatting a volume, but on Macs the default is case-insensitive.

iCloud

As far back as the early 2000s, Apple was experimenting with first-party cloud services baked into macOS. Before iCloud came iTools, .Mac, and MobileMe. The names changed, but the core idea was always syncing data across devices.

We won't dive into how iCloud works here. What matters is that using it is optional, but it greatly simplifies synchronization between Apple computers and devices.

The most prominent and important part of iCloud is iCloud Drive, where users can store documents; the system then automatically tracks changes and syncs them to the cloud. There's also the CloudKit framework, which developers use to synchronize settings and other non-document data.

iCloud is closely tied to your Apple ID, but they're not the same thing. Strictly speaking, you don't need an Apple ID to use a Mac, but without one you can't download apps from the App Store. An Apple ID also includes a small amount of free iCloud storage—currently 5 GB.

Security

SIP

Historically, on Macs an administrator account had the same capabilities as root—the Unix superuser. After confirming a command with their password, they could, for example, modify or delete any file on the system.

That left an opening for malware: if you could lower the user's guard and trick them into entering their password, you could then covertly do practically anything.

OS X 10.11 El Capitan introduced System Integrity Protection (SIP).
In practice, it's a kernel-level suite of protections that prevents changes to the system itself—even by an administrator.

System files and directories are now protected via a special extended file attribute; code injection into processes is blocked—not just into system processes, but others as well; and the kernel will no longer load unsigned extensions.

Of course, there are times when SIP gets in the way of doing something you need. As someone who likes to tinker with the system, I've been running up against this protection regularly since it was introduced.

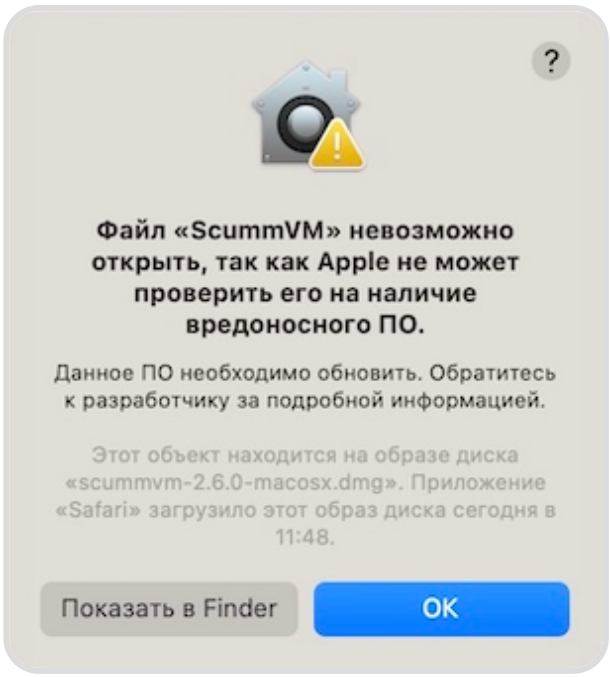
You can disable System Integrity Protection (SIP). Reboot into Recovery mode, open Terminal, and use the csrutil tool. Then reboot once more—and expect two additional restarts later when you re-enable SIP (which is still recommended).

Gatekeeper

Another mechanism designed to protect the Mac from malware is called Gatekeeper. It's a system that relies on Apple's databases to identify potentially malicious files and block them from running.

Here's how it works: Every downloaded file is placed in quarantine and tagged with an extended attribute. When you try to run a quarantined executable, the system checks whether it's on a blocklist, whether it's signed by a certified developer, and, if so, whether the file's contents have been altered.

Depending on the check results, the system either runs the file immediately or refuses to do so. If the issue is a missing signature, it's easy to bypass—just open the file from the right-click context menu instead of double-clicking it.



If you want, you can completely disable Gatekeeper with a single command—no reboot required. While that might make things easier, it's not recommended. Apps don't end up on blocklists for no reason!

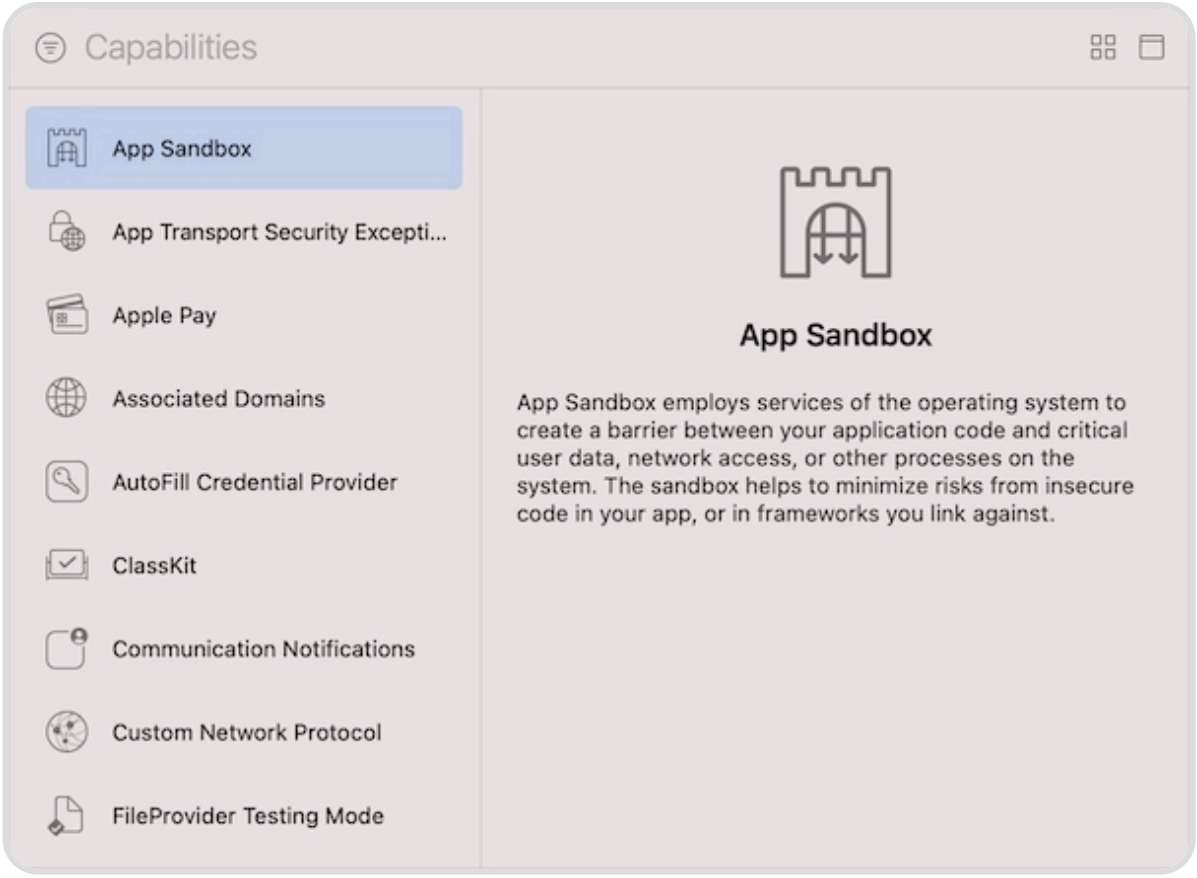
Secure Enclave

On Macs with M-series chips, or with Intel processors paired with T-series coprocessors, Apple uses a dedicated hardware security component—the Secure Enclave—for securely storing and attesting/validating encryption keys and biometric data.

The Secure Enclave is invisible to the user—you can't interact with it directly. Instead, the system communicates with it whenever you confirm a payment, authenticate with your fingerprint, or unlock your computer with an Apple Watch.

App Sandbox

Independent of SIP and Gatekeeper, there's the App Sandbox mechanism, which controls third-party apps' access to user and system files and folders. More precisely, it lets developers themselves restrict what their apps can do. For apps distributed via the App Store this is mandatory; for software obtained through other channels, it's optional.



Configuring the App Sandbox in Xcode

When enabling App Sandbox in Xcode during a build, the developer chooses which capabilities the app may need: access to specific folders, network protocols, and features like Apple Pay. At runtime, the app will prompt the user to grant permission to use each requested resource.

FairPlay

Beyond mechanisms meant to protect users' data, macOS also includes a system that protects data from the users. This is FairPlay—Apple's implementation of digital rights management (DRM). It's arguably the only major restriction that's hardwired into macOS.

FairPlay can be applied to individual MP4 video and AAC audio files, as well as to streaming via FairPlay Streaming. Notably, Netflix uses this DRM.

FairPlay tightly restricts copying of protected content, so if you try to take a screenshot of a movie from Netflix or Apple TV, you'll just get a black rectangle. Recording the video or capturing audio from a protected source won't work either without additional workarounds.

Application Development

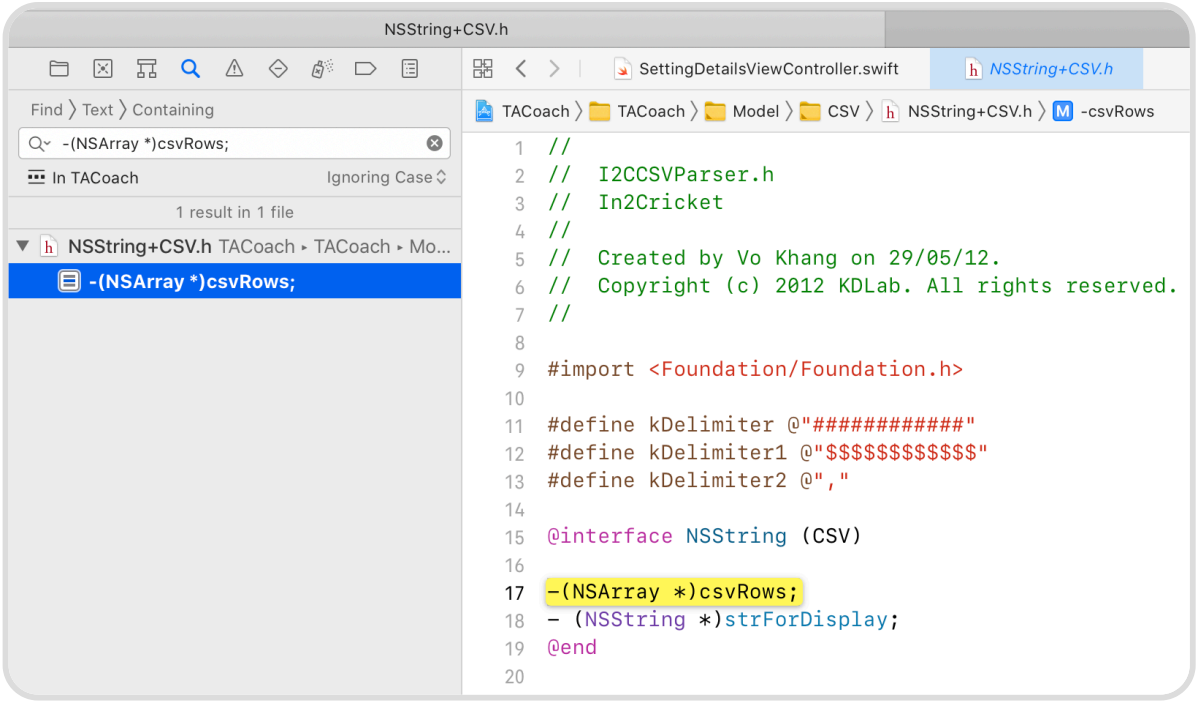
Even at a glance, it's clear that most macOS apps share a consistent visual style, with standard Open and Save dialogs, font and color pickers, and other common components. That's because they're built on the standard object libraries that implement the Aqua interface.

This approach not only ensures a consistent look and predictable app behavior, but also reduces RAM usage and allows Apple to make changes at any time. For example, when dark mode was introduced in macOS 10.14 Mojave, adopting it required minimal effort from app developers.

That said, it's no longer accurate to say macOS has just one windowing library. In practice, there are multiple frameworks: some are retained for backward compatibility, while others are still experimental.

Objective-C

The traditional—and still the most reliable—way to build macOS software is to use the Objective-C language and the Cocoa framework, specifically AppKit (known as NSApp back in the NeXTSTEP era).



Objective-C development in Xcode

Objective-C is a high-level, object-oriented language that supports multiple typing styles. It borrows from Smalltalk (objects and message passing) and from C (basic syntax, direct/manual memory management). Conceptually, you can think of it as Apple's counterpart to C++—and in some ways, to C# and Java.

AppKit

Knowing Objective-C alone isn't enough to build macOS apps.
The bigger piece is the array of system-supplied APIs and frameworks. You'll almost immediately run into the Foundation framework, which provides core objects and data types, and, when building graphical apps, AppKit.

AppKit implements the familiar MVC (Model–View–Controller) pattern: the model defines the data structure, the view handles its presentation, and the controller responds to user actions. With AppKit, you define the appearance and behavior of all windows, menus, buttons, text fields, and other interface elements.

When building AppKit-based apps, developers often use Interface Builder, which ships with Xcode. It's a visual UI design tool that lets you assemble an app from components, like building with blocks (if Visual Basic, Delphi, C++Builder, or Qt Designer come to mind, you're on the right track). Many developers criticize this approach, arguing that it favors proficiency with Interface Builder over actual programming skills.

Swift

Objective-C has served NeXT and Apple faithfully since the 1980s and did its job well. However, over all those years it changed very little and lacks many of the features modern developers take for granted.

At WWDC 2014, Apple announced a new language for building Mac apps: Swift. It was created by Chris Lattner, known as one of the lead developers of the LLVM compiler infrastructure.

Like Objective-C, Swift is a compiled language designed for building stable, high-performance applications. It also uses C-like syntax, but borrows some paradigms from scripting languages such as JavaScript and Python.

In terms of capabilities, Swift can already serve as a full replacement for Objective-C and even outperforms it in performance. However, not all core frameworks have been rewritten in Swift yet, so developers of complex apps still regularly have to deal with Objective-C and its legacy.

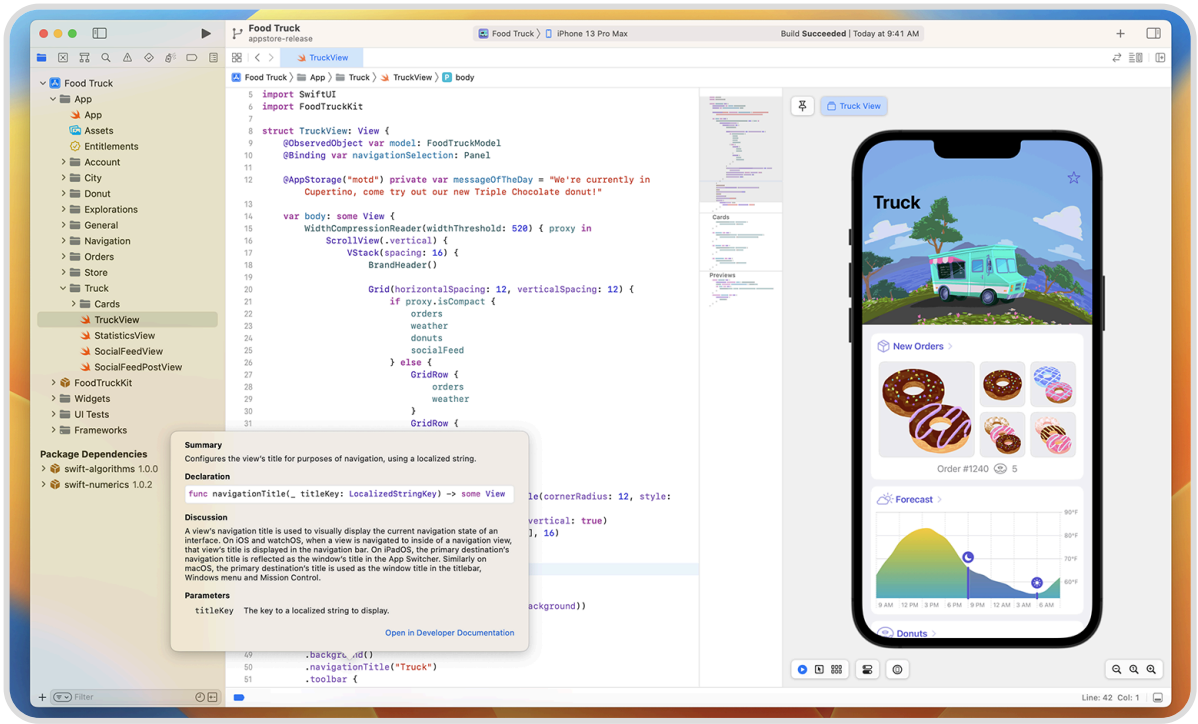
> **INFO**
> Swift is open source, and there are **versions** of the compiler and REPL for Linux, Windows, and Android. However, the language has yet to gain widespread adoption beyond macOS.

SwiftUI

NSApp/AppKit was built alongside Objective-C and tailored to its capabilities. While you can access it from Swift, the framework that naturally pairs with the new language is SwiftUI.

SwiftUI is built on UIKit (Cocoa Touch), a newer offshoot of AppKit created for iOS. It's intended to replace both frameworks and lets you build apps that run across all Apple platforms: watches, phones, tablets, Macs, and Apple TV.

The key advantage of SwiftUI is that it lets you build apps in a declarative style. Instead of issuing step-by-step commands about which UI elements go where, you simply describe what the app should look like, and SwiftUI takes care of the layout and all the intermediate steps. As a result, you no longer need Interface Builder.



Building a UI in SwiftUI

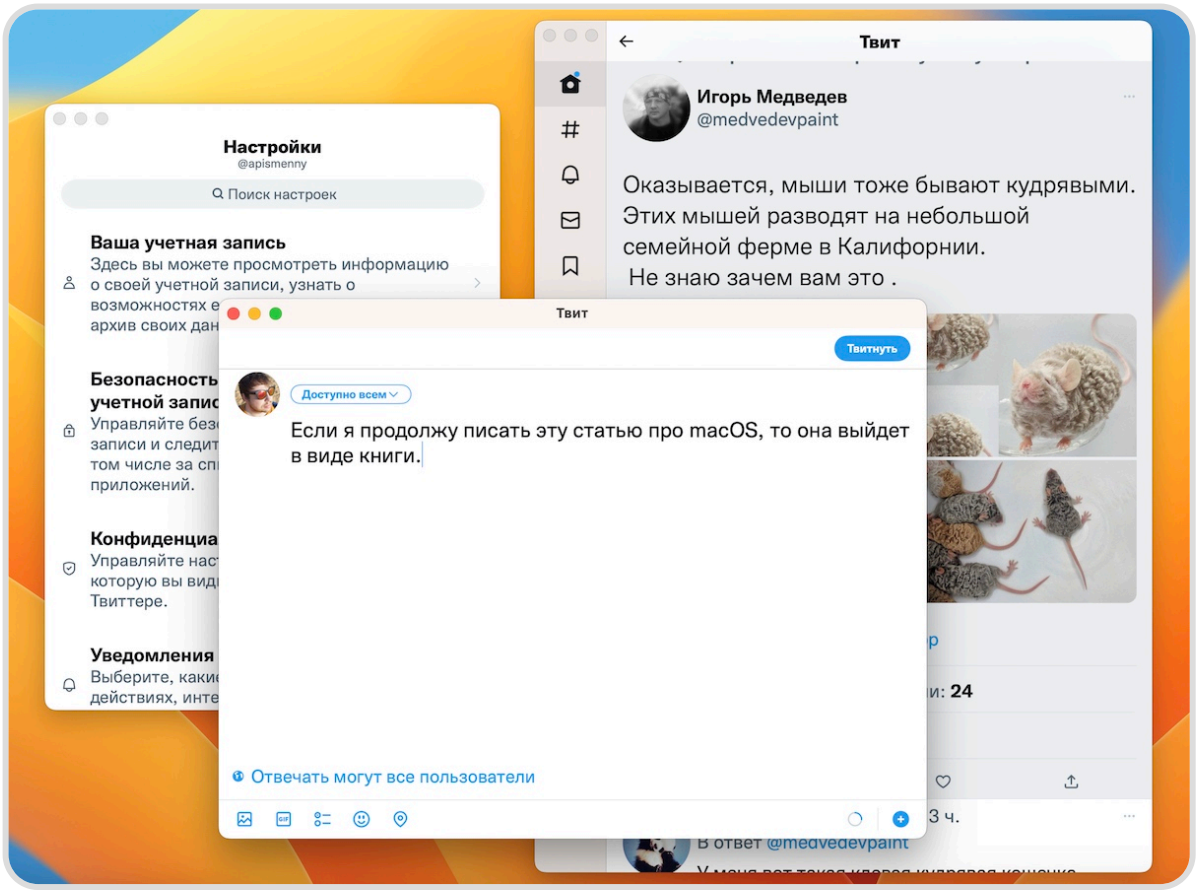
SwiftUI lets developers save a lot of effort when your goals align with what it supports. Otherwise, SwiftUI can become a serious obstacle. It may get easier as SwiftUI evolves, but for now you'll constantly run into

cases Apple didn't anticipate, which push you back to AppKit—fortunately, nothing stops you from mixing it with SwiftUI.

Catalyst

Another modern Apple framework worth mentioning is Catalyst. It's designed to streamline porting iOS apps built with UIKit to the Mac. According to Apple, you can turn a mobile app into a desktop app in as little as a day.

To prove the point, they demoed the official Twitter client ported from iOS. And… you can actually use it, if you want. However, its behavior is noticeably different from truly native Mac apps.



Twitter client built with Catalyst

Catalyst is essentially a stopgap framework, and over time it's expected to be fully replaced by the more universal SwiftUI. In practice, only

a handful of apps were ported to the Mac with Catalyst, and it will likely be retired sooner than later.

Carbon

The Catalyst story is reminiscent of another episode in Apple's history. When Mac OS X was brand new and short on apps, Apple decided to help developers of Classic Mac OS software migrate to the new system.

That's how the Carbon framework was born: it provided support for legacy APIs while still presenting the familiar Aqua interface to users. Carbon proved a major boon for developers, and it was used, among others, in Adobe products—right up until the release of Creative Suite 5.

> **INFO**
> It can seem like you're only allowed to use the standard Apple tools to build apps on the Mac. That's not the case: plenty of popular apps are built with cross-platform libraries like GTK+, Qt, wxWidgets, or the Electron framework (which powers, for example, VS Code and Slack). Apple users simply tend to prefer native apps because they integrate better with the system.

WebObjects

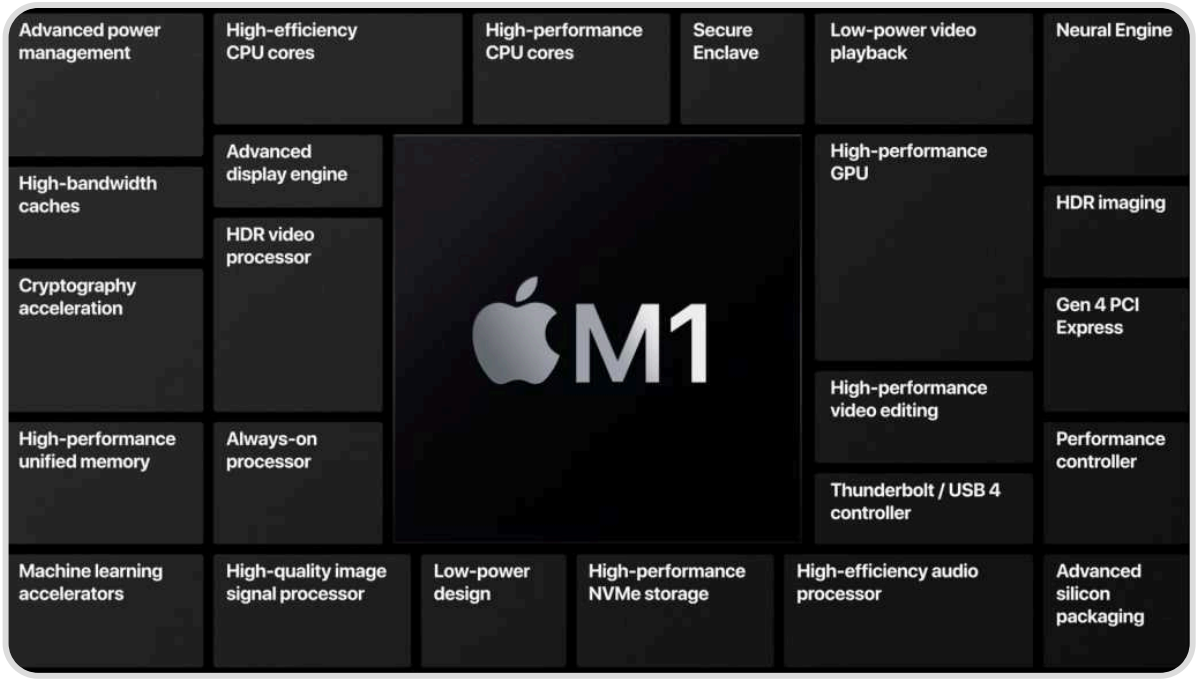
Finally, you might also come across the term WebObjects. That's the name of Apple's object-oriented web framework, originally from the NeXT era and built in the early days of the internet. It was far ahead of its time in many ways, but running a Mac as a web server was a strange idea back then—and still is. The only company that still uses

WebObjects is Apple itself. Parts of the App Store and Apple Music are built on this technology.

Apple Silicon

Apple takes vertical integration seriously: not just building its own software, but also hardware that's a perfect fit for that software.
The choice of CPU is a critical strategic decision. Apple has switched Mac CPU architectures three times: from Motorola 68000 (32-bit CISC) to PowerPC (32- and 64-bit RISC), then to Intel (32- and 64-bit CISC), and in 2020 to ARM (64-bit RISC).

By the time of the most recent transition, Apple was already shipping iPhones, iPads, and other devices with its ARM-based A-series system-on-chips. So moving the Macs to the same hardware platform wasn't unexpected. People had been anticipating it for years, and the only real surprise was how almost completely painless—and even invisible—the change was for users.



Apple Silicon Macs (that's Apple's official name for their ARM-based chips) retained compatibility with nearly all applications—primarily thanks to the binary translation system known as Rosetta 2.

On the first launch of an executable built for Intel-based Macs, Rosetta 2 translates x86 instructions into ARM, i.e., it performs ahead-of-time (AOT) compilation. When necessary, just-in-time (JIT) compilation is also available.

All of this lets you run the same applications as before, even if their developers haven't rebuilt them for the new architecture. In fact, in many cases performance not only matches that of Macs with x86 chips—it actually surpasses it.

Another factor that made the Apple Silicon transition smooth and largely seamless was the groundwork laid years in advance. Apple deftly dropped support for legacy 32-bit apps a year before announcing Apple Silicon, in macOS 10.15 Catalina. Rosetta 2 doesn't run those apps, and that extra year gave third-party developers time to adapt.

As for the apps and frameworks bundled with macOS, Apple began adding 64-bit support back in 2009 with Mac OS X 10.6 Snow Leopard. The Carbon libraries, which never had 64-bit versions, were discontinued in 2012 with OS X 10.8 Mountain Lion.

These systematic housecleaning efforts let Apple switch hardware platforms with ease and keep legacy support only during transition periods.

Miscellaneous

Beyond the items already mentioned, macOS includes several components that bring it closer to the Linux world. For example, it uses the CUPS printing system and a VNC server for screen sharing. And of course, there's the Bash shell, long the de facto standard on Linux (with Zsh set as the default in recent macOS releases).

For many years, macOS bundled interpreters for Python, Perl, and PHP. In macOS 12 Monterey, Apple removed Python 2.7 and PHP, leaving only Perl; however, installing Xcode or the Command Line Tools for Xcode also installs Python 3.

The reason for the "cleanup" is that these languages are mostly used by developers and enthusiasts—and typically in versions much newer than what the system provides. Since you'll have to install a newer version anyway, the old one just gets in the way.

> **INFO**
> macOS also includes the Apache web server, but that's mostly a historical footnote. Up through 10.8 Mountain Lion you could manage Apache from System Preferences. Now it's only accessible from the command line, and it will likely be removed in a future release for the same reasons Apple dropped Python 2.

Key takeaways

So, I think you've already realized that macOS isn't Linux, but it's much closer to it than, say, Windows.

It uses its own kernel, graphics stack, windowing environment, and GUI frameworks. POSIX compliance makes it possible to port Unix-compatible applications to macOS. Some of those are already bundled with the system.

As for the rumors about how "closed" macOS is, they're largely exaggerated. It's no more closed than Windows—and in some areas, considerably less so. On the other hand, it's clearly not Linux, where

almost every component has multiple variants and you can spin up a new fork if you want. And of course, it's widely known that macOS isn't officially allowed to be installed on anything other than Macs.

Criticism of these facts somehow morphed into rumors that on macOS you can't listen to music unless you bought it from the iTunes Store, or run apps downloaded from the internet. That's complete nonsense, of course.

There's a concern that macOS will gradually be merged with the much more locked-down iOS and iPadOS. To see Apple's strategy here, just look at SwiftUI: it lets you build a single app that adapts its look and behavior across devices based on their size, capabilities, and usage patterns. Clearly, Apple isn't trying to eliminate those differences—in fact, they're more likely to keep emphasizing the distinctions between different classes of computers.

So on the one hand, a full-blown merger is unlikely; on the other, it's already happening. It's just happening at a different layer: they're unifying under-the-hood systems (e.g., APFS), building a common foundation for app development (SwiftUI), and macOS is gradually picking up new features from iOS (and vice versa—iOS and iPadOS are getting some things from the Mac).

For many years, macOS has been an excellent choice for everyday work, creative projects, and many types of development. Apple's ongoing optimization and housekeeping help maintain performance and deliver innovations across every layer of the system. On top of that, macOS offers tremendous automation potential—something we've written about more than once and will revisit in the future.